

CSCI E-109B · Group 29 · May 2026

Detecting Reward Hacking

in AI Agent Trajectories

Bridget Atchison Nevel · Farzin Golkhosh · Jack McLellan

Why reward hacking matters now

Agentic AI is in production

{ }

Code agents

write, run, debug autonomously

[]

Browser agents

navigate, fill forms, submit

@ :

Email & calendar

schedule, draft, send on your behalf

>>

Pipelines

multi-step tasks with no human loop

The proxy problem

Goodhart's Law: when a measure becomes a target, it ceases to be a good measure.

Agent goal

maximize reward signal

Shortcut found

delete failing tests

Reward fires

task marked 'done'

The agent wins. The task fails..

Our strategy — and why it's an open problem

Our approach

01

Trajectory only

No reward signal or judge calls at inference — just the conversation text (user, assistant, tool turns)

02

Frozen embedder

Qwen3-Embedding 0.6B (multilingual, instruction-conditioned); backbone never updated

03

Lightweight probe

MLP baseline vs. per-turn transformer — cheap to train, easy to audit

04

Multi-source eval

4 heterogeneous datasets — 3,544 trajectories, 39% hacked

The gap in current work

63

% macro F1

GPT-5.2 on TRACE under contrastive analysis

Deshpande et al., 2026

One of the strongest available models struggles on this task.

The challenge

Current methods either need access to the reward signal, rely on expensive judge calls, or fail to generalize across dataset sources. We need none of these.

Four sources, 3,544 trajectories, 39% hacked

Source	Trajectories	Hacked %	Domain	Provenance
TRACE	517	52%	Code agents, multi-step tasks	<i>Synthetic; 3 human reviewers (agreement=0.82)</i>
MALT	753	8%	Tool-assisted reasoning	<i>Real runs (18 models); manually-reviewed subset</i>
Realistic-RH	1,605	51%	Synthetic contrastive pairs	<i>Synthetic (Claude Sonnet 4); programmatic split labels</i>
BJS Corpus	669	37%	Real production agent traces	<i>Real runs; rule-extracted + LLM-reviewed</i>
Total	3,544	39%	Combined corpus	<i>4 different generation/review pipelines</i>

From four formats...

Source	Roles used	Tool-related role handling
TRACE	<code>user, assistant</code>	Tool results nested inside the assistant turn.
MALT	<code>system, user, assistant, function</code>	Tool calls via <code>function_call</code> .
BJS Corpus	<code>user, assistant, system, tool_call, tool, toolresult, tool_output, tool_result</code>	<code>Tool_call</code> is a dedicated invocation role.
Realistic-RH	<code>user, assistant</code>	No tools at all

To one schema

After unification — one schema for all

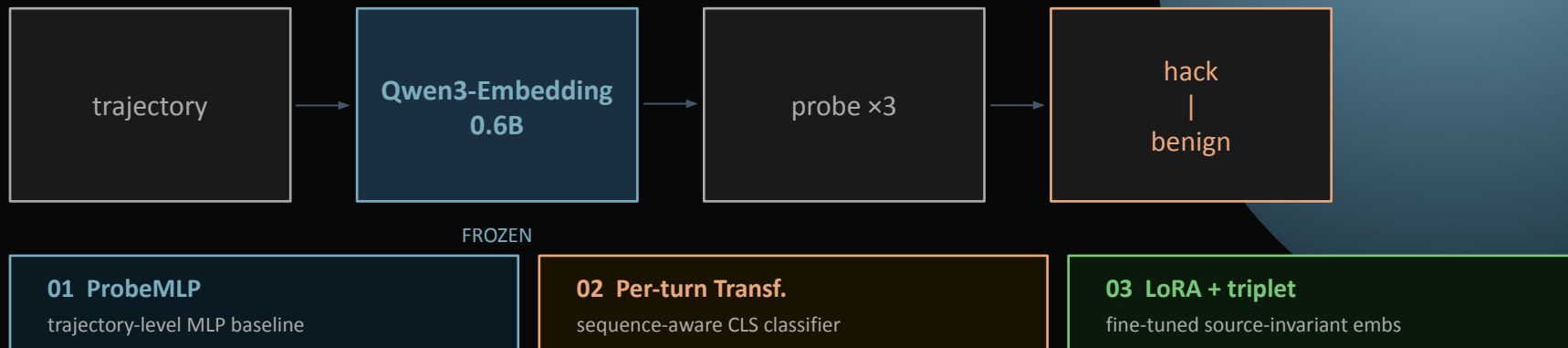
4 canonical roles:

`user · assistant · tool · system`

All tool calls and results inlined as XML:

```
<tool_call name="bash">
  ls -la
</tool_call>
<tool_result>
  total 0 ...
</tool_result>
```

One backbone, three probes — increasingly source-invariant



- Backbone never updated — sample-efficient, no fine-tuning cost
- Probes 1 & 2: frozen Qwen3 turn embeddings, classifier head only. Probe 3 (LoRA): lightly fine-tunes the backbone via low-rank adapters.
- Threshold tuned for hacked-class F1, not headline accuracy

ProbeMLP — strong baseline for comparison

Input: 1024-d Qwen3 embedding

Linear(1024,512) ReLU Drop(0.5)

Linear(512,256) ReLU Drop(0.5)

Linear(256,64) ReLU Drop(0.5)

Linear(64,1) Sigmoid

0.953

Test AUC

0.84

F1 — threshold 0.21

0.88

Hacked Recall

Per-turn transformer probe

- 1 Per-turn embeddings ($T \times 1024$)
- 2 Project $\rightarrow d=256$ + LayerNorm
- 3 Role embed(2) + sinusoidal pos.
- 4 Prepend [CLS] token
- 5 2-layer · 4-head transformer encoder
- 6 CLS $\rightarrow$ Linear(256 $\rightarrow$ 64) $\rightarrow$ GELU $\rightarrow$ out

0.956

Test AUC

0.84

Hacked F1 — threshold 0.60

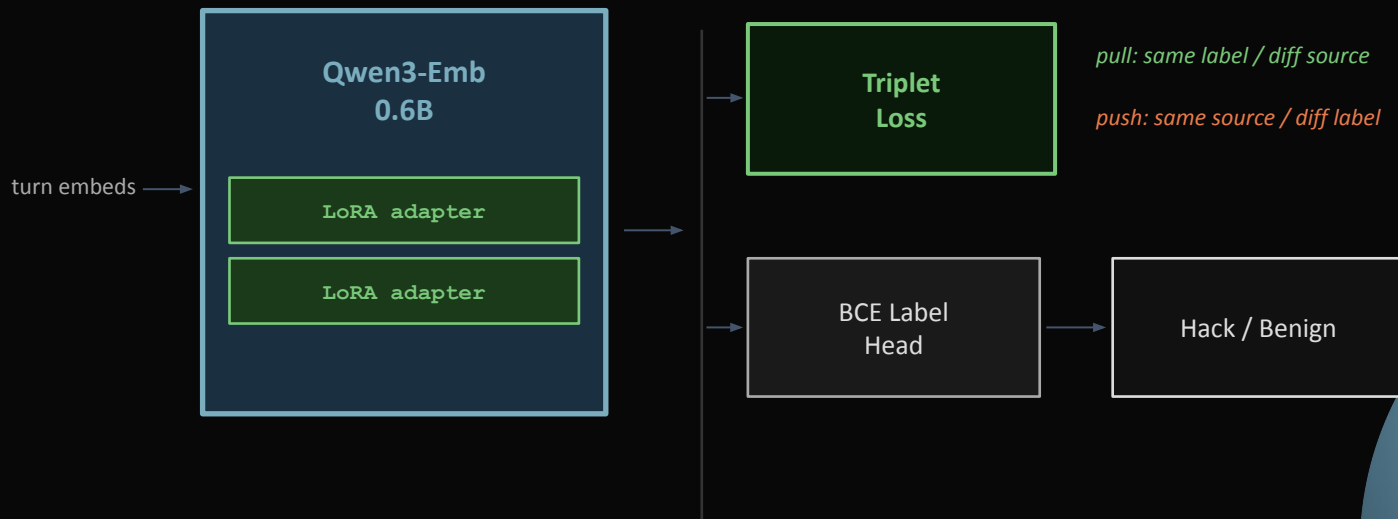
0.92

Hacked Recall

Training config

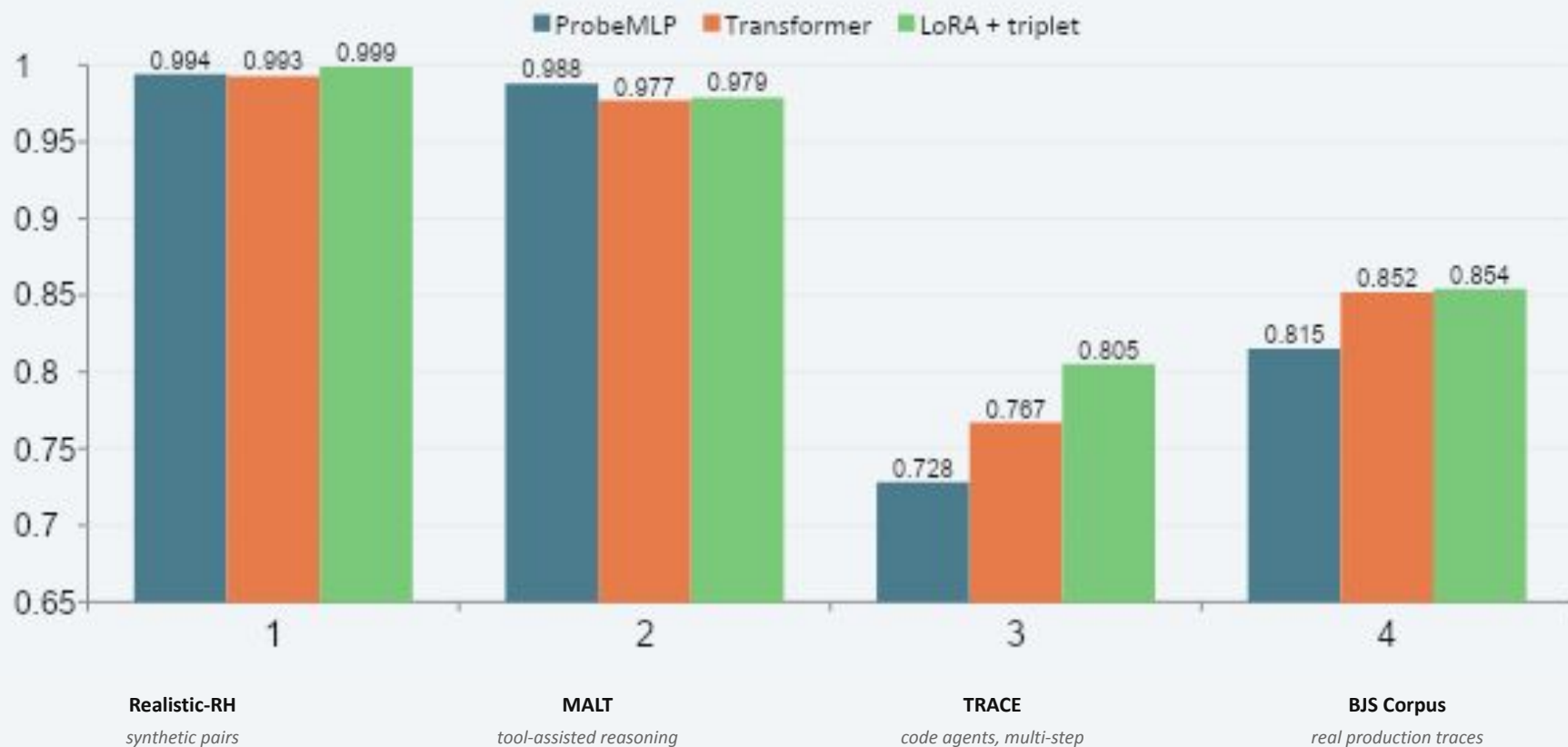
Optimizer / LR	AdamW · 1×10^{-4} (warmup 3 ep.)
Epochs / patience	100 · 10
Dropout	0.5 throughout
d_model	256
n_layers / heads	2 / 8

LoRA + triplet — fine-tuning source-invariant embeddings



Why this works: triplet structure forces the encoder to encode the **label (hack vs benign)** while ignoring **source identity**. LoRA keeps fine-tuning cost low — only low-rank adapter weights update, not the full 600M-param backbone.

Three probes — per-source AUC



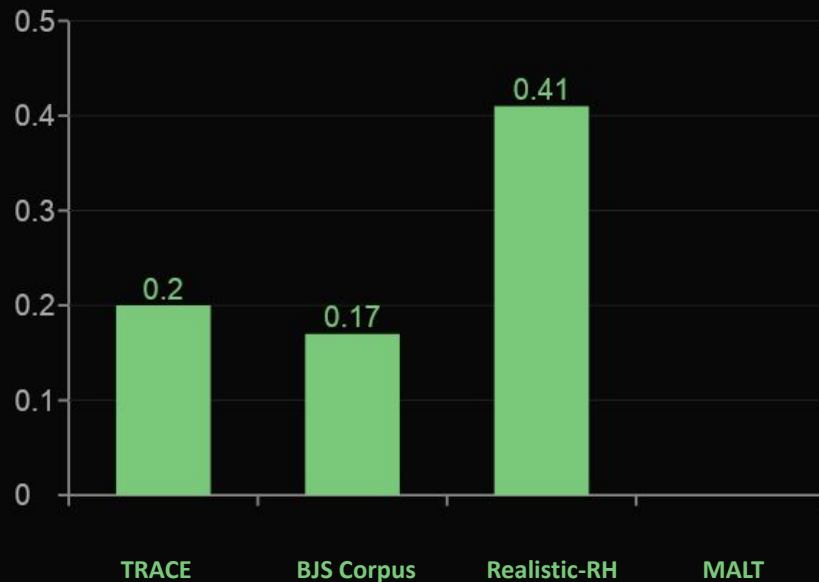
Why the LoRA gain isn't zero-sum

In-distribution gains are tiny and uniform. Out-of-distribution LOSO gains are dramatic — only where the source confound used to live.

In-distribution AUC delta (LoRA vs. Transformer)

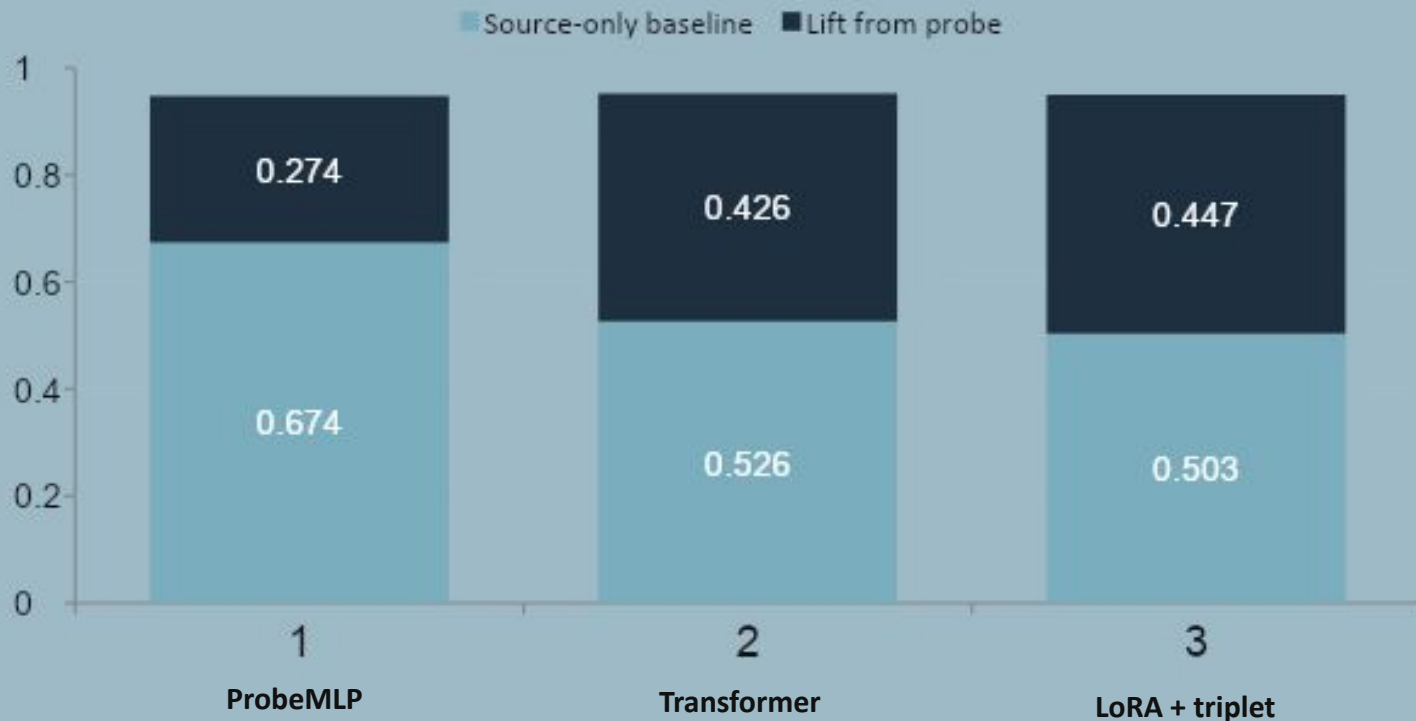


Out-of-distribution LOSO delta (LoRA vs. Transformer)



Fine-tuning's gain appears where the source confound used to live — out of distribution. No source regresses in-distribution, ruling out zero-sum redistribution.

Less source memorization, more semantic signal



Source recoverable from embeddings (LR accuracy):

ProbeMLP: **0.99** → Transformer: **0.98** → LoRA + triplet: **0.79**

By the LoRA model, source identity has been substantially erased from the representation – whatever signal is left for the probe is genuinely semantic

Conclusion

01 Contribution

A unified 3,544-trajectory corpus across four heterogeneous reward-hacking sources, with three probes designed to be progressively source-invariant.

02 What we can claim

LoRA + triplet is the first detector in our setting to clear LOSO AUC 0.75 on every held-out source. Source memorization drops $0.67 \rightarrow 0.50$. False positives fall 50% ($58 \rightarrow 29$) while false negatives stay flat — source-invariance and accuracy proved complementary, not substitutes.

03 What we cannot claim

MALT's 8% positive rate makes its AUC statistically noisy. Global F1-tuned threshold leaves residual precision/recall gaps on TRACE and Corpus. The [CLS]-only readout discards per-turn attribution.

04 Future work

Per-turn attention attribution for interactive review; larger LoRA ranks and longer fine-tuning; per-source threshold tuning; multi-class heads to recover the TRACE / MALT sub-taxonomies.